

Step 2 — A Supervisor Routes Between Them — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume Step 1's `notes_agent.py` (`run_notes_agent`) and `web_agent.py` (`run_web_agent`) already work standalone.

Basic Version

Approach 1 — the direct way, routing decided with plain `if` statements

```
# state.py
from typing import TypedDict
```

```
class TeamState(TypedDict):
    task: str
    needed: list[str]
    visited: list[str]
    notes_findings: str
    web_findings: str
```

```
# supervisor.py
from langgraph.types import Command
```

```
def supervisor_node(state):
    needed = state.get("needed", [])
    if not needed:
        needed = ["notes_agent"]
        if "http" in state["task"].lower():
            needed.append("web_agent")

    visited = state.get("visited", [])

    if "notes_agent" in needed and "notes_agent" not in visited:
        return Command(goto="notes_agent", update={"needed": needed})
```

```
if "web_agent" in needed and "web_agent" not in visited:  
    return Command(goto="web_agent", update={"needed": needed})  
return Command(goto="__end__")
```

This works for the two test tasks that need exactly one specialist, and for task 3, which needs both. It's missing a helper for deciding `needed` cleanly (it's inline and a little repetitive), and `visited` update logic that lives outside `supervisor.py` — both fine for a first working version.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a named helper for "what does this task need," reused cleanly

```
# supervisor.py
from langgraph.types import Command
from state import TeamState

def figure_out_needed(task: str) -> list[str]:
    """Decide which specialists a task needs, based on simple keyword checks."""
    needed = []
    task_lower = task.lower()
    if "note" in task_lower:
        needed.append("notes_agent")
    if "http" in task_lower or "fetch" in task_lower or "page" in task_lower:
        needed.append("web_agent")
    if not needed:
        needed.append("notes_agent")
    return needed

def supervisor_node(state: TeamState) -> Command:
    needed = state.get("needed") or figure_out_needed(state["task"])
    visited = state.get("visited") or []

    for specialist in needed:
        if specialist not in visited:
            return Command(goto=specialist, update={"needed": needed})

    return Command(goto="__end__")
```

```
# notes_agent.py (adds the node function; run_notes_agent unchanged from Step 1)
from langgraph.types import Command
from state import TeamState

def notes_agent_node(state: TeamState) -> Command:
    result = run_notes_agent(state["task"])
```

```
visited = state.get("visited", []) + ["notes_agent"]
return Command(goto="supervisor", update={"notes_findings": result, "visited":
```

```
# web_agent.py (adds the node function; run_web_agent unchanged from Step 1)
from langgraph.types import Command
from state import TeamState

def web_agent_node(state: TeamState) -> Command:
    result = run_web_agent(state["task"])
    visited = state.get("visited", []) + ["web_agent"]
    return Command(goto="supervisor", update={"web_findings": result, "visited":
```

```
# graph.py
from langgraph.graph import StateGraph
from state import TeamState
from supervisor import supervisor_node
from notes_agent import notes_agent_node
from web_agent import web_agent_node

def build_graph():
    graph = StateGraph(TeamState)
    graph.add_node("supervisor", supervisor_node)
    graph.add_node("notes_agent", notes_agent_node)
    graph.add_node("web_agent", web_agent_node)
    graph.set_entry_point("supervisor")
    return graph.compile()
```

```
# main.py
from graph import build_graph

if __name__ == "__main__":
    app = build_graph()
    initial_state = {
        "task": "Check my notes for anything about Project Atlas, and also fetch",
        "needed": [],
        "visited": [],
        "notes_findings": "",
        "web_findings": "",
    }
    final_state = app.invoke(initial_state)
    print("Visited:", final_state["visited"])
```

```
print("Notes findings:", final_state["notes_findings"])
print("Web findings:", final_state["web_findings"])
```

Expected output (abridged, order of `visited` may vary):

```
Visited: ['notes_agent', 'web_agent']
Notes findings: Your notes say Project Atlas is targeted for a Q3 launch...
Web findings: The status page reports Project Atlas is on track and 80% complete.
```

Difference from Basic: the routing decision is now a named, reusable `figure_out_needed()` function instead of inline logic mixed into the node, and both specialist nodes follow the exact same small shape — run the Step 1 function, append their own name to `visited`, write their own field, route back to `"supervisor"`. That symmetry matters: it means adding a third MCP-backed specialist later would mean copying this same 4-line pattern, not inventing a new one.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — using `langgraph.graph.END` explicitly, and testing routing logic on fake state first

```
# supervisor.py
from langgraph.graph import END
from langgraph.types import Command
from state import TeamState

def figure_out_needed(task: str) -> list[str]:
    needed = []
    task_lower = task.lower()
    if "note" in task_lower:
        needed.append("notes_agent")
    if "http" in task_lower or "fetch" in task_lower or "page" in task_lower:
        needed.append("web_agent")
    if not needed:
        needed.append("notes_agent")
    return needed

def supervisor_node(state: TeamState) -> Command:
    needed = state.get("needed") or figure_out_needed(state["task"])
    visited = state.get("visited") or []

    for specialist in needed:
        if specialist not in visited:
            return Command(goto=specialist, update={"needed": needed})

    return Command(goto=END)
```

```
# test_supervisor_routing.py
# Tests the routing DECISION against a hand-built fake state -- no real
# agent calls, no MCP connections -- so a routing bug is easy to place,
# separate from "do the real specialists work."
from supervisor import supervisor_node

# task needing only notes, nothing visited yet -> should route to notes_agent
```

```

state1 = {"task": "What do my notes say?", "needed": [], "visited": [], "notes_fi
result1 = supervisor_node(state1)
assert result1.goto == "notes_agent", result1.goto

# task needing both, notes already visited -> should route to web_agent next
state2 = {
    "task": "Check my notes and fetch the status page.",
    "needed": ["notes_agent", "web_agent"],
    "visited": ["notes_agent"],
    "notes_findings": "some findings",
    "web_findings": "",
}
result2 = supervisor_node(state2)
assert result2.goto == "web_agent", result2.goto

# task needing both, both visited -> should route to END
state3 = {
    "task": "Check my notes and fetch the status page.",
    "needed": ["notes_agent", "web_agent"],
    "visited": ["notes_agent", "web_agent"],
    "notes_findings": "some findings",
    "web_findings": "some web findings",
}
result3 = supervisor_node(state3)
from langgraph.graph import END
assert result3.goto == END, result3.goto

print("All routing tests passed.")

```

Expected output:

```
All routing tests passed.
```

Approach 2 — full graph run, confirming both specialists actually ran once each

```

# main.py
from graph import build_graph

TASKS = [
    "What do my notes say about Project Atlas?",
    "Fetch https://intranet.example.com/atlas/status and tell me what it says.",
    "Check my notes for anything about Project Atlas, and also fetch its status p
]

if __name__ == "__main__":

```

```

app = build_graph()
for task in TASKS:
    print(f"\n=== Task: {task} ===")
    final_state = app.invoke({
        "task": task, "needed": [], "visited": [], "notes_findings": "", "web
    })
    print("Visited:", final_state["visited"])
    if final_state["notes_findings"]:
        print("Notes findings:", final_state["notes_findings"])
    if final_state["web_findings"]:
        print("Web findings:", final_state["web_findings"])

```

Expected output (abridged):

```

=== Task: What do my notes say about Project Atlas? ===
Visited: ['notes_agent']
Notes findings: Your notes say Project Atlas is targeted for a Q3 launch...

=== Task: Fetch https://intranet.example.com/atlas/status and tell me what it say
Visited: ['web_agent']
Web findings: The status page reports Project Atlas is on track and 80% complete.

=== Task: Check my notes for anything about Project Atlas, and also fetch its sta
Visited: ['notes_agent', 'web_agent']
Notes findings: Your notes say Project Atlas is targeted for a Q3 launch...
Web findings: The status page reports Project Atlas is on track and 80% complete.

```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate proves the whole graph works end to end, but only by running it and reading the output. Approach 1 separates two concerns Project 4's Step 5 also separated: is the *routing decision* correct (tested here against fake state, no LLM calls, no MCP connections, runs in milliseconds), versus does the *full graph with real specialists* work (Approach 2). If Approach 2 ever produces a wrong final state, Approach 1's tests tell you immediately whether the bug is in the routing logic or somewhere inside a specialist's own code — the same value Project 4's "test routing against fake state first" step description called out directly.

Which one should you actually write? Both. Keep Approach 1's fake-state routing tests as a fast, cheap first check any time you touch `supervisor.py` — they catch a routing bug in milliseconds, before you've spent real API calls and real MCP connections finding out the hard way. Then run Approach 2's full graph for the real end-to-end proof. This two-layer testing habit — logic tested in isolation, then the real system tested end to end — is worth carrying into Step 3 and Step 4 too, where a routing bug would otherwise be much harder to spot underneath a real Writer call.